

Dense Proximity Reward Produces Reactive PPO Policies in Atari Breakout*

Mike Harrell
mikey.harrell@gmail.com

August 2026

Abstract

Proximal Policy Optimization (PPO) agents trained on deterministic Atari environments routinely converge to memorized action sequences (fixed scripts that replay the same buttons in the same order regardless of game state) rather than developing reactive, ball-tracking policies. Across 123 controlled experiments spanning every major environment-modification approach (sticky actions, entropy tuning, auxiliary cursor-alignment objectives, dynamics randomization, adversarial obstacles, and frame skipping), no model produced a genuinely reactive argmax on Atari Breakout. We identify a fundamental confound underlying these failures: every diagnostic in the standard RL toolbox measures the policy distribution, not the argmax, so models routinely appear reactive while evaluating to deterministic scripts. We introduce the *split-watcher*, a diagnostic that measures the argmax directly by running the same model on two different brick layouts with independent per-side predictions, and show that it detects memorization where distribution-level diagnostics fail. We then demonstrate that a compact dense proximity reward, which rewards the paddle for horizontal closeness to the ball during descent, produces the first verified reactive PPO argmax on Atari Breakout. The model achieves 0/240 perfect transfers on the split-watcher (100% no-timing ALT retention), an intervention gradient AUC of 0.421, and a stochastic best score of 216 on clean evaluation. Directly rewarding the desired behavior succeeds where penalizing memorized behavior fails: PPO optimizes whatever is in the reward function. Making ball-tracking explicitly rewarding, rather than trying to make scripts non-viable through environment engineering, is what ultimately works.

1 Introduction

Proximal Policy Optimization (PPO) [1] is one of the most widely used deep reinforcement learning algorithms, and the Arcade Learning Environment (ALE) [2] remains a standard benchmark, popularized by the DQN agent [11] which demonstrated human-level play across dozens of Atari games. However, a well-known issue with training on deterministic ALE games is *policy memorization*: rather than learning to react to visual input, the agent learns a fixed action sequence (a “script”) that exploits deterministic environment dynamics to achieve a consistent score without any genuine perception of game state [3, 4].

The standard mitigation is sticky actions (repeat_action_probability=0.25), proposed by Machado et al. [3] and widely adopted as part of the ALE evaluation protocol. However, Zhang et al. [4]

*Code and reproduction: <https://github.com/mharrell/breakout-reactive-ppo>. Full project history: <https://github.com/mharrell/BreakoutBot>.

showed that sticky actions do not prevent memorization in deep ConvNet-based RL agents: convolutional networks are “automatically robust” to 25% action noise, and a memorized sequence survives with only minor score degradation.

This paper reports the results of a systematic investigation into policy memorization in PPO agents on Atari Breakout. Over 123 controlled experiments spanning every major proposed environment-modification mitigation, we found that *no* such approach produced a reactive argmax. Every model that appeared reactive under standard diagnostics was revealed as a memorized script when the argmax (rather than the policy distribution) was tested directly. A 124th experiment, using a fundamentally different strategy (dense behavioral reward instead of environment engineering), succeeded.

The contributions of this work are:

1. We identify a fundamental confound in the standard RL diagnostic toolkit: every commonly used metric measures the *policy distribution*, not the *argmax action*, and distribution-level reactivity routinely coexists with argmax memorization.
2. We introduce the *split-watcher*, a simple diagnostic that measures argmax behavior directly by running the same model on two different initial states with independent per-side predictions.
3. We show that a compact dense proximity reward (directly rewarding the paddle for being horizontally close to the descending ball) produces the first verified reactive PPO argmax on Atari Breakout, achieving 0/240 perfect transfers on the split-watcher.

Experiment naming. Throughout this paper, individual PPO training runs are referenced by numbered labels (PPO.1 through PPO.124). These represent 124 controlled single-variable experiments conducted over the course of this investigation. PPO.1–106 explored sticky actions, entropy tuning, reward clipping, and architectural variants. PPO.107–117 tested *cursor wrappers* (auxiliary training objectives that shape the policy distribution toward a target paddle position by adding a cursor-alignment term to the loss, without changing the game reward). PPO.118–123 tested brick layout randomization and transfer to BeamRider, another Atari game. None of these 123 experiments produced a reactive argmax. PPO.124, the proximity-reward model, is the focus of this paper.

2 Related Work

2.1 Memorization in Deterministic Environments

The problem of policy memorization in deterministic RL environments has been recognized since the early days of deep RL on the ALE. Machado et al. [3] proposed sticky actions (repeat_action_probability=0.25) as a standard mitigation and documented the prevalence of deterministic exploitation across Atari games. However, Zhang et al. [4] directly tested whether sticky actions prevent memorization in DQN and other deep RL agents, finding that “stochasticity could neither prevent deep RL agents from serious overfitting nor detect overfitted agents effectively.” The present work independently confirms Zhang et al.’s finding across PPO agents: every sticky-trained model collapsed to a deterministic script when tested without sticky actions.

Subsequent work has explored various approaches to forcing reactive behavior. Justesen et al. [5] showed that agents can exploit deterministic patterns even in procedurally generated environments. Cobbe et al. [6] demonstrated that Procgen’s procedural generation improves generalization but does not eliminate it. The Go-Explore algorithm [7] achieved superhuman performance on Montezuma’s Revenge but did so through explicit state-space exploration, not reactive policy learning.

2.2 Dense Auxiliary Rewards

Adding auxiliary rewards to guide learning has a long history in RL. Ng, Harada, and Russell [8] showed that *potential-based* reward shaping preserves the optimal policy (a desirable property when the goal is to accelerate convergence without changing what the agent learns). In deep RL, auxiliary tasks and dense rewards have been used to improve sample efficiency and final performance [9]. The present work uses an auxiliary reward for the opposite purpose: rather than preserving the original optimum while accelerating learning, we deliberately add a non-potential-based reward to *change the optimum itself*, making ball-tracking the locally reward-maximizing behavior during training rather than trying to make scripts non-viable through environment engineering.

2.3 Diagnosing Memorization

Standard evaluation protocols for Atari RL rely on aggregate metrics (mean episode reward, unique score count) that do not distinguish reactive behavior from memorized scripts. The memorization check used in this project’s earlier work ¹ counts unique deterministic scores across evaluation episodes, but we found that score variance from life-loss timing and ball-bounce stochasticity produces false MULTIPLE_SCRIPTS verdicts for confirmed-memorized models. The split-watcher introduced here is, to our knowledge, the first diagnostic that directly measures argmax behavior rather than properties of the policy distribution or score distribution.

3 Method

3.1 The Split-Watcher Diagnostic

The split-watcher runs the same trained model on two different Breakout layouts simultaneously, making independent predictions for each side. The left side presents a full brick wall; the right side presents an altered layout (half the bricks cleared, or a different arrangement). Both sides use the same model weights and the same argmax action selection.

Different brick layouts cause different ball bounces. On the altered side, the ball hits different bricks at different angles and arrives at the paddle from different positions. A reactive policy that tracks the ball *must* move differently on the two sides. A memorized script produces identical action sequences regardless.

We compute the Pearson correlation of paddle X-positions between the two sides:

$$\text{px_corr} = \frac{\text{cov}(\mathbf{p}_{\text{full}}, \mathbf{p}_{\text{alt}})}{\sigma_{\text{full}}\sigma_{\text{alt}}} \tag{1}$$

A $\text{px_corr} > 0.99$ with ALT score $\approx$ FULL score is classified as a *perfect transfer*: definitive memorization, physically impossible for a reactive policy.

We evaluate across three layout types:

- **RIGHT_HALF**: bricks 0–17 cleared (right half of the wall)
- **LEFT_HALF**: bricks 18–35 cleared (left half of the wall)
- **RANDOM_50**: 50% of bricks cleared at random

¹Full project history and methodology at <https://github.com/mharrell/BreakoutBot>

We also provide a *no-timing variant* that removes the NoopResetEnv wrapper (which adds 0–30 random NOOP frames at reset), eliminating timing offsets as a confound. Both sides start at frame 0 identically; any divergence comes purely from different brick layouts.

Full code for the split-watcher is available in the companion repository at <https://github.com/mharrell/breakout-reactive-ppo>.

3.2 Intervention Gradient

As a complementary diagnostic, we developed the *intervention gradient* test. Mid-game, after the ball has been in play for sufficient time for the policy to establish a trajectory, the ball’s horizontal position (ALE RAM address 99) is teleported by a displacement $\Delta x \in \{0, \pm 8, \pm 15, \pm 30, \pm 45, \pm 60\}$ pixels. We record whether the paddle’s next action reverses direction to follow the teleported ball. A memorized script ignores the teleport and continues its fixed sequence. A reactive tracker reverses direction to track the new ball position.

A center-hold dead script (a policy that holds the paddle at the center of the screen regardless of ball position) serves as the calibration baseline. The reversal rate is plotted against displacement magnitude to produce a dose-response curve; the area under this curve (AUC) quantifies overall reactivity.

3.3 Proximity Reward Wrapper

The proximity reward wrapper reads three ALE RAM addresses at each timestep: paddle X (72), ball X (99), and ball Y (101). When the ball is descending toward the paddle ($\text{ball_y} > 100$), a bonus is computed:

$$\text{bonus} = 0.05 \cdot \max \left(0, 1 - \frac{|\text{paddle_x} - \text{ball_x}|}{80} \right) \quad (2)$$

The bonus ranges from 0.0 (paddle and ball separated by ≥ 80 pixels, half the playfield width) to 0.05 (paddle centered directly under the ball). At $\sim 2,000$ descent frames per game, perfect tracking earns approximately 50 bonus points, roughly 5–10% of a typical game score and equivalent to 7 yellow bricks. The scale is intentionally small: large enough to steer the policy toward ball-tracking over the course of a game (thousands of frames per episode), small enough not to overwhelm the game’s natural reward signal (1.0–7.0 per brick).

The proximity bonus is applied *during training only*. Evaluation and memorization checks use clean Breakout (no wrapper), testing whether the tracking behavior transfers to the original reward function.

3.4 Training Configuration

The wrapper ordering in Table 1 is load-bearing: `ClipRewardEnv` clips per-step rewards to their sign ($\{-1, 0, +1\}$), so the 0.05 proximity bonus would be rounded to zero if `ProximityRewardWrapper` were applied before it. Placing `ProximityRewardWrapper` after `ClipRewardEnv` ensures the bonus is added post-clip and reaches the optimizer unmodified. The full wrapper stack matches the companion repository’s `train.py`.

Parameter	Value
Algorithm	PPO (Stable-Baselines3 [10])
Architecture	NatureCNN
Environments	32 (DummyVecEnv)
n_steps	128
Batch size	1024
n_epochs	4
γ	0.99
Entropy coefficient	0.006
Learning rate	$2.5 \times 10^{-4} \rightarrow 10^{-5}$ (linear)
Clip range	$0.2 \rightarrow 0.05$ (linear)
Total steps	25,000,000
Environment	ALE/Breakout-v5, frameskip=4, no sticky actions
Training wrappers	NoopResetEnv, FireResetEnv, EpisodicLifeEnv, GrayscaleResize(84×84), ClipRewardEnv, ProximityRewardWrapper (applied <i>after</i> ClipRewardEnv)
Eval wrapper	EpisodicLifeEnv (clean Breakout, no proximity reward)
Hardware	Single NVIDIA RTX 3060 Ti, Windows 11
Training time	~6.5 hours for 25M steps

Table 1: Training configuration.

4 Results

4.1 Split-Watcher Verification

Table 2 shows the split-watcher results. Across 240 games (120 no-timing, 120 with NoopResetEnv), the model produced **zero perfect transfers**. In the no-timing variant, ALT score retention was 100%: the model cleared every brick on every layout, every game. With NoopResetEnv, retention dropped to 46–59%. The 0–30 random NOOP frames added at reset corrupt the initial frame stack (four stacked frames may span the serve, causing the model to miss the ball’s initial trajectory) and shift the episode’s starting phase relative to any serve-timing patterns learned during training. Whether the drop reflects purely a perceptual artifact (frame-stack corruption) or partial memorization of serve timing is unresolved; both mechanisms predict the observed sensitivity. The improvement from 46% to 59% between checkpoints is consistent with increasing robustness under either account.

The mean px_corr of 0.94–0.96 is below the 0.99 perfect-transfer threshold but still indicates substantial correlation between paddle trajectories on different layouts. A fully reactive policy may retain nontrivial px_corr because the ball’s gross left-right motion is similar across layouts during the opening serve and early bounces; the action-level divergence (62–71%) is the more direct measure of behavioral adaptation. For comparison, memorized cursor-wrapper models (PPO_111–118) and two BeamRider-trained models (another Atari game where the same distribution-vs-argmax confound was independently observed; see Experiment Naming above) all produced at least one perfect transfer (px_corr > 0.99) under the split-watcher. PPO_124 is the first model in the project’s history to achieve zero perfect transfers across all test conditions.

Checkpoint	Games	Perfect Transfers	px_corr	ALT Retention	Divergence
<i>No-timing variant (no NoopResetEnv)</i>					
best (19.2M)	60	0	0.943	100%	62.4%
final (25M)	60	0	0.959	100%	62.9%
<i>With NoopResetEnv (0–30 frame offset)</i>					
best (19.2M)	60	0	0.945	46%	71.4%
final (25M)	60	0	0.961	59%	70.6%

Table 2: Split-watcher results for PPO_124. px_corr is the mean Pearson correlation of paddle X-positions across the three layout types (RIGHT_HALF, LEFT_HALF, RANDOM_50). Perfect transfer threshold: px_corr > 0.99 and ALT score $\approx$ FULL score. 0/240 total perfect transfers.

Displacement	Dead Baseline	PPO_124 final
± 0 px	0.0%	37.5%
± 8 px	0.0%	41.2%
± 15 px	0.0%	60.0%
± 30 px	0.0%	50.0%
± 45 px	0.0%	31.2%
± 60 px	0.0%	25.0%
AUC	0.000	0.421

Table 3: Intervention gradient: reversal rate by displacement magnitude. Dead baseline is a center-hold script that ignores the teleport.

4.2 Intervention Gradient

Table 3 shows the dose-response curve. The $\Delta x = 0$ condition is a control: the ball is “teleported” to its current position (i.e., not moved), measuring the model’s natural rate of paddle direction changes during normal play. A reactive tracker changes direction 37.5% of the time simply because the ball moves; this is the baseline, not noise. The dead baseline (center-hold script) never changes direction, producing 0.0% reversal at all magnitudes.

Relative to the $\Delta x = 0$ baseline, the net intervention effect is strongest at ± 15 px (+22.5 percentage points above baseline), declines to +12.5pp at ± 30 px, and becomes negative at ± 45 –60px. The model reverses direction to follow moderate displacements but does not chase a ball that appears in physically impossible positions. This is the signature of a reactive policy. The dead baseline confirms that the test correctly identifies non-reactive behavior at every magnitude.

The AUC of 0.421 is computed from raw reversal rates and should be interpreted with the 37.5% natural-reversal baseline in mind. Sample sizes are approximately 16–20 moving teleports per magnitude (teleports where the paddle was in motion pre-intervention); rates should be interpreted as exploratory given the small denominators. Prior cursor models (PPO_107–117) showed 33–50% reversal rates on the intervention gradient but were confirmed memorized by the split-watcher; those models had reactive *distributions* but memorized *argmax modes*.

4.3 Memorization Check

Table 4 shows the memorization check results (deterministic inference on clean Breakout, no proximity reward). The check runs the model deterministically for 20 games and classifies behavior by the number of unique episode scores produced: SINGLE_SCRIPT (1 unique score, definitive memorization) or MULTIPLE_SCRIPTS (>1 unique score, not definitive — score variance from life-loss timing and ball-bounce stochasticity can produce false positives). PPO.124 is the first model in the project’s history to sustain MULTIPLE_SCRIPTS under deterministic inference without sticky action masking. Of the 13 checkpoints in the 14M–25M range, 10 produced multiple unique scores (Table 4 shows a representative subset). Earlier models achieving MULTIPLE_SCRIPTS did so only with sticky actions enabled (a known confound: sticky noise produces 8–14 unique scores from a dead policy). Under stochastic inference (stochastic) on clean Breakout, the model achieves a best score of 216, the highest recorded in this project without sticky masking.

Checkpoint	Unique Scores	Best Score	Verdict
1M	1	16	SINGLE_SCRIPT
5M	1	37	SINGLE_SCRIPT
10M	1	78	SINGLE_SCRIPT
14M	4	87	MULTIPLE_SCRIPTS
19M	2	107	SINGLE_SCRIPT
25M	4	93	MULTIPLE_SCRIPTS

Table 4: Memorization check: number of unique deterministic scores on clean Breakout.

The best scores in Table 4 (87–107) appear substantially lower than the split-watcher full-wall scores (379–383, Section 4.1). This is not a contradiction: the memorization check uses the standard ALE evaluation setup (NoopResetEnv with 0–30 random NOOP frames at reset), while the no-timing split-watcher removes NoopResetEnv entirely. The same 0–30 frame offset that drops ALT retention from 100% to 46–59% also degrades full-wall performance by a comparable margin. The no-timing variant reveals the model’s capability when frame-stack corruption is eliminated; the standard-eval scores reflect a known sensitivity to initial serve timing. We note that the 19M checkpoint (SINGLE_SCRIPT on memcheck) and the 19.2M checkpoint (0/240 on split-watcher, Table 2) are separated by only 200k steps. The two diagnostics use different environment configurations (with and without NoopResetEnv), so their results are not directly comparable at a given checkpoint. The 19.2M checkpoint’s memcheck result was not recorded; the model may oscillate between reactive and script-like behavior on short timescales, or the discrepancy may reflect the test-protocol difference alone.

4.4 Continuation Training

Extending training from 25M to 50M steps (PPO.126, identical configuration) did *not* improve reactivity. The best checkpoint at 47.4M showed layout-specific partial decoupling (RIGHT_HALF: px_corr=0.33, ALT=223), but the final checkpoint at 50M converged to a single memorized script (px_corr=0.95 on all layouts, ALT=401 every game). Intervention AUC at 50M was 0.327 (vs. 0.421 for PPO.124 final). PPO’s optimizer eventually finds a script that maximizes the combined game + proximity objective; the proximity reward delays but does not permanently prevent script convergence. This suggests that checkpoint selection, rather than training to convergence, is critical for proximity-reward models.

5 Discussion

5.1 Why Proximity Reward Works

Every prior anti-memorization method in this project tried to change the *viability of scripts*: make the environment stochastic so scripts fail (sticky actions, random ball perturbations), add penalties so repetitive sequences are punished (entropy bonuses, trajectory entropy), or make the environment geometrically complex (moving bumpers, brick randomization). PPO’s optimizer always found a script that survived the changes: timing-robust, layout-conditioned, or noise-tolerant. The optimum was always a script; only the shape changed.

Proximity reward changes *the training-time optimization landscape*. A center-hold script (the default PPO convergence point for Breakout) receives incidental proximity bonus when the ball happens to pass near center. But a reactive tracking policy receives the maximum bonus on every single descent frame, earning 3–5× more proximity reward over a full game. During early-to-mid training, the gradient toward ball-tracking is unambiguous and strong: track the ball, get more reward. The proximity bonus makes tracking the locally dominant behavior for tens of millions of steps.

However, PPO_126’s regression at 50M steps (Section 4.4) shows that the asymptotic optimum of the combined game + proximity objective can still be a script: one that positions the paddle where the ball is statistically most likely to arrive, earning high proximity bonuses without frame-by-frame reactivity. The proximity reward creates an extended *reactive phase* during training but does not permanently eliminate scripts from the optimization landscape. Checkpoint selection during this phase is critical; the best policy may occur well before convergence.

The bonus is dense (every frame) while the game’s natural reward (brick breaks) is sparse (every few seconds). Dense rewards provide better gradients for credit assignment. The model does not need to discover the multi-step causal chain “track ball → hit ball → ball breaks brick → score.” It is told directly: being near the ball is good. And because the scale is small (0.05 vs. 1.0–7.0 per brick), it does not overwhelm the game reward. The model still cares about breaking bricks; it learns that the way to break bricks is to be under the ball when it comes down.

5.2 The Distribution-vs-Argmax Confound

A critical methodological finding of this work is that *every* commonly used RL diagnostic (memorization check via unique score count, intervention probe via distribution shift after state perturbation, SCAD probe via mutual information between action and state, brick layout test via score retention) measures properties of the *policy distribution*, not the *argmax action*. But evaluation uses the argmax. A model can maintain a reactive-looking distribution (90% LEFT when ball is left, 90% RIGHT when ball is right) while the argmax is always the same action regardless.

This was confirmed empirically: cursor-wrapper models (PPO_107–117) showed 33–50% reversal rates on the intervention gradient and MULTIPLE_SCRIPTS on memcheck, but were definitively memorized under split-watcher verification (perfect transfer on altered layouts). The intervention gradient correctly measures that the distribution $P(a | s)$ changes when the ball position changes. It does not measure whether $\arg \max_a P(a | s)$ changes. For models with high-confidence distributions (e.g., 90% probability on the top action), substantial probability mass can redistribute among the remaining actions without ever crossing the argmax threshold.

The split-watcher is, to our knowledge, the only existing diagnostic that directly measures argmax behavior by comparing paddle trajectories on different initial states. We recommend it as a standard verification step for any claim of reactive behavior in deterministic RL environments.

5.3 Limitations

1. **Single game, RAM-dependent implementation.** This work demonstrates the proximity reward approach only on Atari Breakout. The principle (directly reward the behavior you want) should generalize, but the specific implementation reads privileged ALE RAM addresses (`ale.getRAM()` [72], [99], [101]) to obtain ball and paddle positions. This requires per-game reverse-engineering of RAM layouts and does not extend to games without documented RAM maps or to environments that do not expose ground-truth entity positions. Applying this approach to, e.g., Montezuma’s Revenge would require an entirely different state-access mechanism.
2. **Scale sensitivity.** The scale of 0.05 was found to work; the sensitivity of reactivity to different scales (0.01, 0.10, 0.25) has not been systematically tested.
3. **Finite training window.** PPO_126’s regression at 50M steps shows that the proximity reward delays but does not permanently prevent script convergence. The duration of the “re-active window” and whether it can be extended through scale scheduling remains unexplored.
4. **Single training run.** PPO_124 is a single training run (seed=124). No replicate runs with different seeds were performed, so seed sensitivity is unknown. The 123 prior experiments provide context for the difficulty of the problem, but the success case itself has $N=1$.
5. **No baselines.** No non-PPO agent (DQN, human, oracle ball-tracker) has been evaluated on the split-watcher. The “0/240 perfect transfers” result lacks a reference distribution for what a known-reactive policy’s `px_corr` distribution looks like on these layouts.
6. **No statistical significance testing.** Results are reported as raw counts and percentages without confidence intervals. The 0/240 perfect transfers result is qualitatively clear, but formal statistical treatment would strengthen the conclusions. Intervention gradient reversal rates are reported without per-magnitude sample sizes; these should be interpreted as exploratory.
7. **No ALE sticky evaluation protocol.** The standard ALE evaluation protocol (sticky actions enabled during evaluation) was not used here, as sticky actions are a known memorization masker. Clean evaluation (no sticky actions) provides the most direct test of argmax behavior.
8. **BrickClearWrapper bug in prior results.** An early version of the split-watcher contained a bug in `BrickClearWrapper`: the `reset()` method returned the pre-clear observation, causing both sides to see identical frame-1 observations. This bug was present in all split-watcher runs for PPO_111–118 and BeamRider models, and was fixed before PPO_124 testing. The prior models have not been re-tested with the fixed wrapper, so their perfect-transfer counts may be inflated. The qualitative conclusion that those models were memorized is independently supported by single-script memcheck scores and zero intervention reversal on center-hold dead baselines.

5.4 Future Work

The proximity reward approach opens several directions:

- **Generalization to other Atari games.** The mechanism (dense reward for spatial alignment with a game entity) is Breakout-specific but the principle (reward the desired behavior directly) should transfer. Space Invaders: reward horizontal alignment with enemies. Beam-Rider: reward being out of the line of fire. Pong: reward paddle-ball proximity (trivially, since the game already does).
- **Scale scheduling.** Since PPO_126 showed regression at 50M, could a scheduled decrease in proximity scale (start high to establish tracking, reduce to let game reward dominate) produce a stable reactive policy?
- **Combination with auxiliary supervision.** Earlier experiments with auxiliary ball-position regression heads (PPO_102 and PPO_103) showed that NatureCNN can track the ball at 1.9px MAE when directly supervised, but the policy still memorized. Combining proximity reward (which shapes the policy) with auxiliary ball-position supervision (which shapes the features) might produce even stronger reactivity.
- **Formal theory.** Under what conditions does a dense auxiliary reward create a reactive phase during training, and can the reactive optimum be made stable? A theoretical treatment of the optimization landscape for deterministic MDPs with auxiliary behavioral rewards would be valuable.

6 Conclusion

After 123 controlled PPO experiments on Atari Breakout spanning every major proposed environment-modification approach, we found that no such method produced a genuinely reactive argmax. Every diagnostic in the standard RL toolbox measures the policy distribution, not the argmax. This confound led to false positive reactivity claims across multiple experimental generations.

The fix was a compact dense proximity bonus: reward the paddle for horizontal closeness to the ball during descent. This makes ball-tracking explicitly rewarding during training, rather than trying to make scripts non-viable through environment engineering. The resulting model (PPO_124) achieved 0/240 perfect transfers on the split-watcher, 100% no-timing ALT retention, and an intervention gradient AUC of 0.421, the first verified reactive PPO argmax on Atari Breakout. However, continued training (PPO_126, 50M steps) showed regression to a memorized script, indicating that the proximity reward creates a reactive phase rather than a permanent change to the asymptotic optimum. Checkpoint selection, rather than training to convergence, is critical.

PPO optimizes whatever is in the reward function. In a deterministic environment, environment engineering alone changes which script is optimal; adding a dense behavioral reward makes the desired reactive behavior locally dominant during training. Directly rewarding the behavior you want may succeed where penalizing memorization has failed.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [2] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

- [3] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht, and M. Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.
- [4] C. Zhang, O. Vinyals, R. Munos, and S. Bengio. A study on overfitting in deep reinforcement learning. *arXiv preprint arXiv:1804.06893*, 2018.
- [5] N. Justesen, R. R. Torrado, P. Bontrager, A. Khalifa, J. Togelius, and S. Risi. Illuminating generalization in deep reinforcement learning through procedural level generation. In *NeurIPS Workshop on Deep Reinforcement Learning*, 2018.
- [6] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning*, 2020.
- [7] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.
- [8] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, 1999.
- [9] M. Jaderberg, V. Mnih, W. M. Czarnecki, T. Schaul, J. Z. Leibo, D. Silver, and K. Kavukcuoglu. Reinforcement learning with unsupervised auxiliary tasks. In *International Conference on Learning Representations*, 2017.
- [10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [11] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.